A

Project Synopsis

On

Quoosh

(Real-Time Quiz Platform)

Submitted to

Savitribai Phule Pune University

In Partial Fulfillment of

Bachelor of Computer Application (Science)

BCA (Science) III Year (Semester II)

Submitted by

Mr. Sanchet Bhauso Kolekar

Mr. Sumit Dattatray Ithape

Department of Computer Application

MIT Arts, Commerce and Science College Alandi (D) Pune

2025 – 2026

MAEER'S
MIT | Arts, Commerce & Science College
Affiliated to Savitribai Phule Pune University
Accredited by NAAC with "A" Grade

CERTIFICATE

This is to certify that the project entitled **"Quoosh - Real-Time Quiz Platform"** has been successfully completed by:

1. **Mr. Sanchet Bhauso Kolekar** (Seat Number:)
2. **Mr. Sumit Dattatray Ithape** (Seat Number:)

This project is submitted in partial fulfilment of the requirements for the course **BCA368: DSE VI Project Laboratory** towards the degree of Bachelor of Computer Application (Science) [BCA (Science)] III Year (Semester II) to Savitribai Phule Pune University during the academic year 2025 – 2026.

The project work has been carried out under our supervision and guidance. To the best of our knowledge, the work presented in this synopsis is original and has not been submitted elsewhere for the award of any other degree or diploma.

Date: _____

Place: Pune, Maharashtra

Instructor **H.O.D./Coordinator**

Internal Examiner **External Examiner**

ACKNOWLEDGEMNET

We feel great pleasure in expressing our deepest sense of gratitude and sincere thanks to our guide, **Dr. Madhuri A. Darekar**, for their valuable guidance during the Project work, without which it would have been a very difficult task. We have no words to express our sincere thanks for the valuable guidance, extreme assistance, and cooperation extended to all the Staff Members of our Department.

This acknowledgement would be incomplete without expressing our special thanks to **Mr. Amit V. Tale**, Head of the Department (Computer Application), for their support during the work.

We are also deeply thankful to our Project Lab Instructor, **Mr. Shivaji Arun Shinde**, for providing us with the necessary technical assistance, practical insights, and the laboratory resources that were crucial for developing and testing Rozgar Chowk.

We would also like to extend our heartfelt gratitude to our Principal, **Dr. Balasaheb B. Waphare**, who provided a lot of valuable support, mostly being behind the veils of college bureaucracy.

Last but not least, we would like to thank all the Teaching and Non-Teaching staff members of our department, our parents, and our colleagues who helped us directly or indirectly in completing this Project successfully.

Mr. Sanchet Bhauso Kolekar

Mr. Sumit Dattatray Ithape

INDEX

Sr. No.	Content	Page No.
1	**Introduction**	1 – 2
	1.1 Problem Statement	
	1.2 Project Objectives	
2	**Requirement Analysis**	2 – 5
	2.1 Functional Requirements	
	2.2 Non-Functional Requirements	
	2.3 Feasibility Study	
	2.4 Performance Requirements	
3	**System Design**	6 – 19
	3.1 System Architecture Diagram	
	3.2 Data Dictionary	
	3.3 ER Diagram	
	3.4 Data Flow Diagram	
	3.5 UI Mock-ups / User Interface Design	
4	**Operating Environment**	20 – 22
	4.1 Hardware Environment	
	4.2 Software Environment	
5	**Deployment Architecture**	22 – 25
	5.1 Web Application (Vercel)	
	5.2 Real-Time Game Engine (Railway)	
	5.3 Database (Supabase)	
	5.4 Deployment Architecture Diagram	
	5.5 Advantages of this Deployment Architecture	
6	**Drawbacks and Limitations**	25
7	**Future Scope**	26
8	**Conclusion**	26
9	**Bibliography**	27
	9.1 Research Papers & Academic Sources	
	9.2 Internet Links	

1. Introduction

Quoosh is a full-stack, real-time multiplayer quiz platform inspired by popular interactive quiz systems such as Kahoot! and Quizizz. It is engineered as a scalable, browser-based solution that enables quiz creators referred to as Hosts to build and manage quizzes, and allows participants to compete live in timed, gamified question rounds through any modern web browser, with no installation required.

The platform is developed as a modern pnpm monorepo containing three tightly integrated packages: a Next.js 16.1.5 frontend with a REST API layer (@quoosh/web), a standalone Socket.IO v4 real-time game server (@quoosh/socket), and a shared common library (@quoosh/common) that defines the typed communication contract between packages. A PostgreSQL 17 database managed through Prisma ORM v5.22.0 on Neon's managed cloud platform powers persistent data storage for users, quizzes, and session records, ensuring full data integrity.

Quoosh bridges the gap between static quiz authoring tools and live competitive gameplay by combining a rich administrative backend with a high-throughput, event-driven game engine, all within a single unified product. The platform is deployed and publicly accessible at **https://quoosh.vercel.app.**

1.1 Problem Statement

Traditional classroom assessments and competitive quiz formats suffer from significant operational and engagement limitations. Paper-based quizzes are time-consuming to administer and evaluate, while existing digital tools often require dedicated hardware, institutional licenses, or lack real-time competitive elements.

From an Information Technology perspective, three critical gaps exist in the current ecosystem:

- **Lack of Real-Time Interactivity:** Existing quiz tools operate asynchronously, providing no live competitive element. There is no infrastructure for broadcasting synchronized game states, tracking live player scores, or dynamically responding to all players answering simultaneously.
- **Absence of a Unified Management Platform:** Quiz creators lack a single platform to create, manage, AI-generate, and host quizzes while also tracking session history, managing player reports, and configuring platform-wide settings through an administrative interface.
- **Scalability and Security Gaps:** Most lightweight quiz tools lack structured role-based access control, secure credential management, content moderation mechanisms, and the architectural separation required to scale the real-time game engine independently from the web application.

1.2 Project Objectives

Quoosh is proposed as a centralized, full-stack web platform designed to deliver real-time multiplayer quiz experiences. The system replaces manual assessment workflows with a structured digital environment where Hosts manage quizzes through a secure dashboard and Players participate through a live, gamified interface.

The primary objectives of the proposed system are:

- To provide a structured quiz creation environment supporting manual entry, AI-assisted generation via Google Gemini, and JSON file-based import.
- To implement a real-time, event-driven game engine capable of synchronizing game state across all participants simultaneously using Socket.IO v4 WebSocket communication.
- To enforce role-based access control (RBAC) distinguishing Host, Admin, and Player roles with appropriate permissions and protected routes.
- To persist all session data including player answers, scores, and session history in a relational PostgreSQL database via Prisma ORM.
- To provide an administrative interface for platform governance covering user management, content moderation, session oversight, announcements, and platform configuration.
- To support graceful reconnection so that disconnected Hosts and Players can resume an active game session without data loss, using a persistent client ID stored in the browser.
- To develop a modular, monorepo-based architecture that allows independent scaling of the real-time socket server and the web application.

2. Requirement Analysis

2.1 Functional Requirements

a) User Management

- Secure registration and login for Hosts using email and bcrypt-hashed passwords (cost factor 12).
- Google OAuth 2.0 integration for one-click sign-in.
- Role-based access control (HOST, ADMIN) enforced at both middleware and API levels.
- JWT-based session management with a 7-day expiry policy.
- Admin-only account suspension and management capabilities.

b) Host Module

- Dashboard displaying all quizzes created by the logged-in Host.
- Quiz creation with title and subject metadata.
- Question management supporting manual entry, AI-assisted generation via Google Gemini, and JSON file import with validation/preview.
- Per-question configuration: answer options, correct answer index, display time (5–120s), and pre-question cooldown.
- Drag-and-drop question reordering using @dnd-kit.
- Hosting a live game session by receiving a shareable invite code and QR code.
- Live game management: starting the game, viewing real-time player list, kicking players, advancing questions, and displaying the leaderboard.
- Host reconnect support via persistent client ID.

c) Player Module

- Joining a game session via a 6-character invite code at /play with no account required.
- Auto-join via ?pin= URL parameter.
- Username and avatar selection before entering the game lobby.
- Real-time answer submission with time-based point scoring — faster answers earn more points (starting at 1000 points, decreasing proportionally).
- Receiving per-question results: correctness, points earned, and current rank.
- Viewing final leaderboard and podium at game completion.
- Session reconnection support using a persistent client ID (UUID v7) stored in browser local Storage.

d) Admin Module

- Admin-only dashboard accessible at /admin, returning HTTP 404 to all non-admin users (security by obscurity).
- Live session monitoring via a dedicated Socket.IO /admin namespace, authenticated by ADMIN_SOCKET_SECRET.
- Host management: listing, searching, suspending, restoring, and permanently deleting host accounts.
- Quiz moderation: reviewing and actioning player-submitted content reports with admin notes.
- Announcement broadcasting to all registered hosts.
- Platform settings management via a key-value configuration store (Platform Setting model).

- Overview dashboard with aggregate platform statistics (total hosts, quizzes, sessions, players, weekly/monthly session charts).

2.2 Non-Functional Requirements

- Secure authentication and authorization using Auth.js v5 (next-auth@5.0.0-beta.30) and bcrypt password hashing.
- Protection against brute-force attacks through Upstash Redis-backed sliding-window rate limiting.
- Type-safe communication contracts between the web client and socket server via the shared @quoosh/common TypeScript package.
- Responsive browser-based interface accessible without any software installation.
- Real-time event delivery with sub-100ms latency for synchronized gameplay.
- Security headers on all responses: X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy.
- HTML input sanitization on all contact form fields before email interpolation.
- Modular monorepo architecture enabling independent deployment and scaling.
- PostgreSQL schema with normalized relational tables, CUID primary keys, cascade deletes, and referential integrity constraints.

2.3 Feasibility Study

a) Technical Feasibility

- Implementation using Next.js 16.1.5 (App Router), Socket.IO v4.8.3, Node.js 20+, and PostgreSQL 17 with Prisma ORM v5.22.0.
- Monorepo architecture managed via pnpm workspaces enabling shared code and consistent dependency management.
- REST API for persistent data operations; WebSocket (Socket.IO) for real-time game state synchronization.
- AI-assisted quiz generation via Google Gemini through the Vercel AI SDK (@ai-sdk/google).
- TypeScript 5.9.3 with strict type checking across all packages via shared tsconfig.

b) Economic Feasibility

- Entirely open-source technology stack with no licensing costs.
- Deployed on free-tier cloud platforms: Vercel for the web application, Railway for the socket server, Neon for PostgreSQL, Upstash for Redis.
- Minimal infrastructure requirements for small-to-medium scale deployments.

c) Operational Feasibility

- No installation required for players — participation via any modern web browser.
- Intuitive quiz builder with drag-and-drop question reordering and live preview.
- Simple invite-code-based game joining eliminates registration friction for participants.
- Admin panel provides non-technical administrators a structured interface for platform governance

2.4 Performance Requirements

- Real-time game state broadcast to all connected clients within 100ms of a server-side event.
- JWT stateless authentication eliminates session lookup overhead on every API request.
- Prisma ORM queries optimized with selective field inclusion (select/include) and pagination (skip/take).
- Atomic quiz update using Prisma transactions to ensure data consistency.
- Socket server capable of managing multiple independent concurrent game sessions via in-memory Registry.
- Abandoned sessions automatically cleaned up: empty-game detection every 60 seconds, eviction after 5 minutes.
- Image proxy cache headers: public, max-age=3600, s-maxage=3600.

3. System Design

3.1 System Architecture Diagram

Quoosh follows a three-tier distributed architecture augmented by a dedicated real-time communication layer. The system is implemented as a pnpm monorepo with three packages: @quoosh/web (Next.js app), @quoosh/socket (Socket.IO server), and @quoosh/common (shared types and validators).

- **Client Layer:** The browser-based frontend built on Next.js 16.1.5 (React 19.2.4) serves Host dashboards, the Player game view, and the Admin panel. The client communicates with the Application Layer via HTTP REST calls and with the Game Engine via persistent WebSocket connections.

- **Application Layer:** Next.js 16.1.5 Route Handlers provide all 22 REST API endpoints. Auth.js v5 handles JWT-based authentication. Prisma ORM v5.22.0 manages all type-safe interactions with the PostgreSQL 17 database hosted on Neon. Middleware enforces RBAC for Host and Admin routes.

- **Real-Time Game Engine:** A standalone Node.js 20 process running Socket.IO v4.8.3 manages all live game sessions entirely in memory for maximum throughput. It maintains a Registry singleton (in-memory game store) and a Game state machine per session. Quiz data is passed directly from the web API as a JSON object via the manager:hostDirect socket event.

- **Data Layer:** A managed PostgreSQL 17 instance hosted on Neon (Singapore, ap-southeast-1 region) stores all persistent data including users, quizzes, questions, sessions, player answers, reports, announcements, and platform settings.

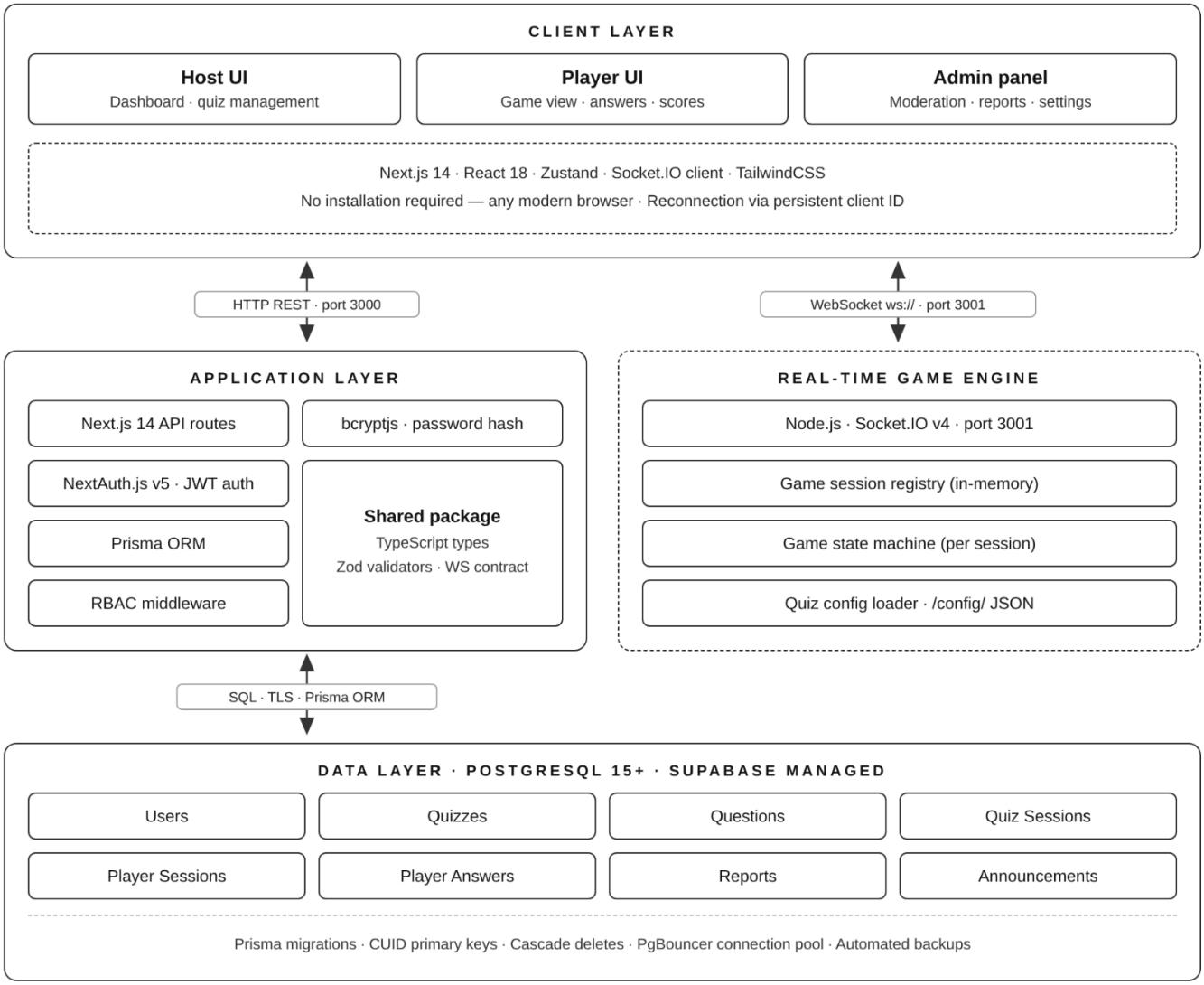

Fig. 3.1 – System Architecture Diagram of Quoosh

3.2 Data Dictionary

The following tables describe all entities in the Quoosh PostgreSQL database schema, their fields, data types, constraints, and descriptions.

Table 3.1: User

Field Name	Data Type	Constraints	Description
id	String (CUID)	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
email	String	UNIQUE, NOT NULL	Login email address
name	String	NOT NULL	Display name of the user
password	String?	NOT NULL	Bcrypt-hashed password
role	Enum (HOST/ADMIN)	DEFAULT HOST, NOT NULL	User role determining access level
suspended	Boolean	DEFAULT false	Whether the account is suspended
createdAt	DateTime	DEFAULT now ()	Account creation timestamp

Table 3.2: Quiz

Field Name	Data Type	Constraints	Description
id	String (CUID)	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
title	String	NOT NULL	Display title of the quiz
subject	String	NOT NULL	Subject / topic of the quiz
status	Enum (QuizStatus)	DEFAULT APPROVED	Moderation status (PENDING / APPROVED / REJECTED)

Field Name	Data Type	Constraints	Description
creatorId	String	FOREIGN KEY → User.id, NOT NULL	References the host who created the quiz
createdAt	DateTime	DEFAULT now ()	Creation timestamp
updatedAt	DateTime	AUTO-UPDATE	Last modification timestamp

Table 3.3: Question

Field Name	Data Type	Constraints	Description
id	String (CUID)	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
quizId	String	FOREIGN KEY → Quiz.id, NOT NULL	References parent quiz
text	String	NOT NULL	The question text
image	String	NULL	Optional image URL
answers	String []	NOT NULL	Array of answer option strings
solution	Integer	NOT NULL	Zero-based index of the correct answer
time	Integer	DEFAULT 20	Answer time window in seconds
cooldown	Integer	DEFAULT 5	Pre-answer display duration in seconds
order	Integer	NOT NULL	Display order within the quiz

Table 3.4: Quiz Session

Field Name	Data Type	Constraints	Description
id	String (CUID)	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
inviteCode	String	UNIQUE, NOT NULL	6-character invite code to join the session
hostId	String	FOREIGN KEY → User.id, NOT NULL	References the hosting user
quizId	String	FOREIGN KEY → Quiz.id, NOT NULL	References the quiz being played
startedAt	DateTime	DEFAULT now ()	Session start timestamp
endedAt	DateTime?	NULL	Session end timestamp
terminated	Boolean	DEFAULT false	Whether session was force-terminated by admin

Table 3.5: Player Session

Field Name	Data Type	Constraints	Description
id	String (CUID)	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
sessionId	String	FOREIGN KEY → QuizSession.id, NOT NULL	References parent quiz session
nickname	String	NOT NULL	Player's chosen display name
avatar	String?	NULL	Player's selected avatar identifier
score	Integer	DEFAULT 0	Final total score
joinedAt	DateTime	DEFAULT now ()	Timestamp when player joined

Table 3.6: PlayerAnswer

Field Name	Data Type	Constraints	Description
id	String (CUID)	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
sessionId	String	FOREIGN KEY → QuizSession.id, NOT NULL	References parent quiz session
playerSessionId	String	FOREIGN KEY → PlayerSession.id, NOT NULL	References the player record
questionId	String	FOREIGN KEY → Question.id, NOT NULL	References the answered question
answerId	Integer	NOT NULL	Index of the answer selected by player
correct	Boolean	NOT NULL	Whether the selected answer was correct
points	Integer	NOT NULL	Points awarded for this answer
answeredAt	DateTime	DEFAULT now ()	Timestamp of answer submission

Table 3.7: Announcement

Field Name	Data Type	Constraints	Description
id	String (CUID)	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
title	String	NOT NULL	Announcement heading
body	String	NOT NULL	Full announcement content
sentBy	String	FOREIGN KEY → User.id, NOT NULL	References the admin who sent it

Field Name	Data Type	Constraints	Description
createdAt	DateTime	DEFAULT now ()	Broadcast timestamp

Table 3.8: Report

Field Name	Data Type	Constraints	Description
id	String (CUID)	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
quizId	String	FOREIGN KEY → Quiz.id, NOT NULL	References the reported quiz
reason	String	NOT NULL	Reason provided for the report
reportedBy	String	NOT NULL	Player nickname or 'anonymous'
status	Enum (ReportStatus)	DEFAULT OPEN	Report status (OPEN / RESOLVED / DISMISSED)
adminNote	String	NULL	Admin's response or action note
createdAt	DateTime	DEFAULT now ()	Report submission timestamp
resolvedAt	DateTime	NULL	Resolution timestamp

3.3 ER Diagram

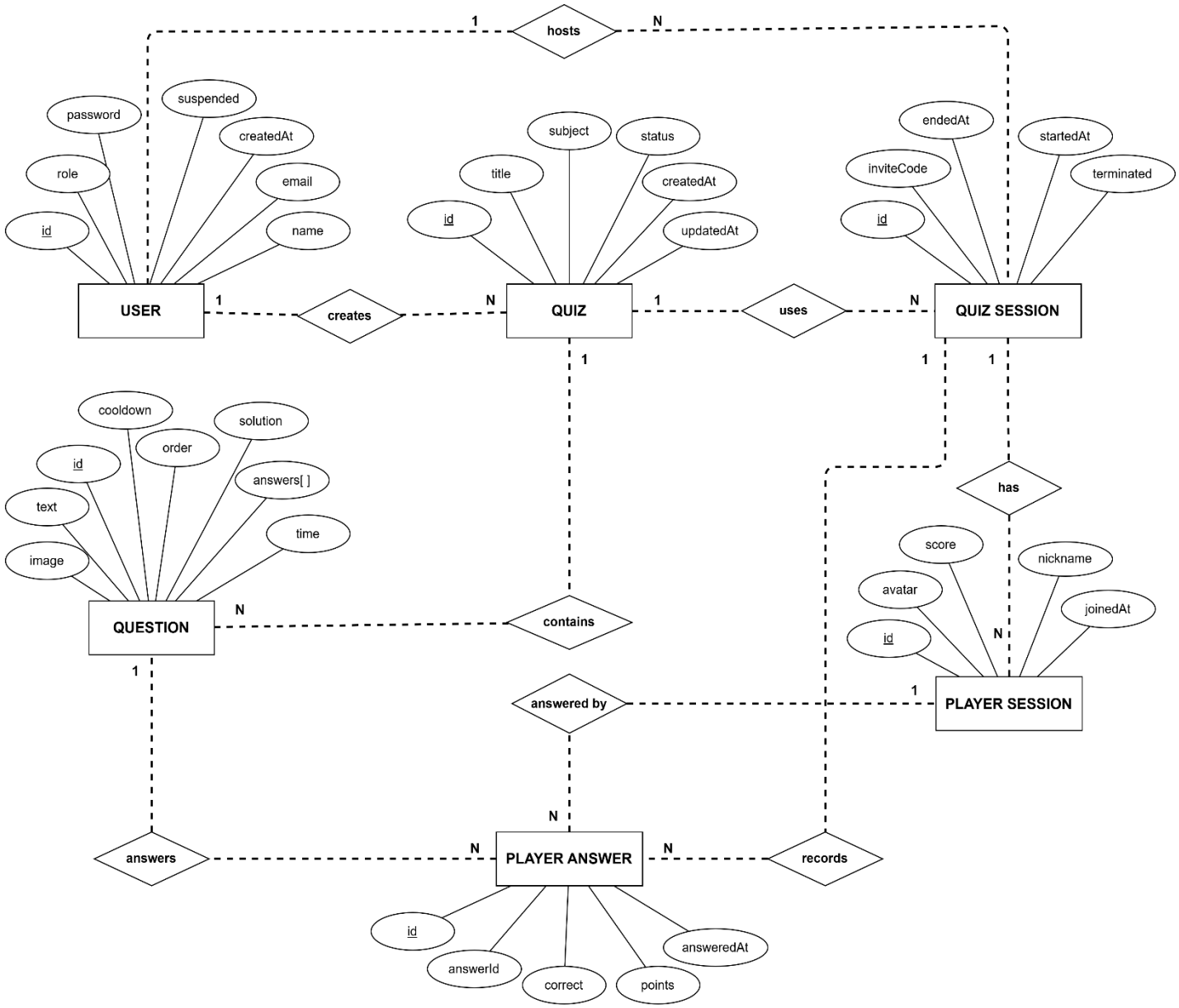

Fig. 3.3 – ER Diagram of Quoosh

3.4 Data Flow Diagram

3.4.1 Context Diagram (Level 0)

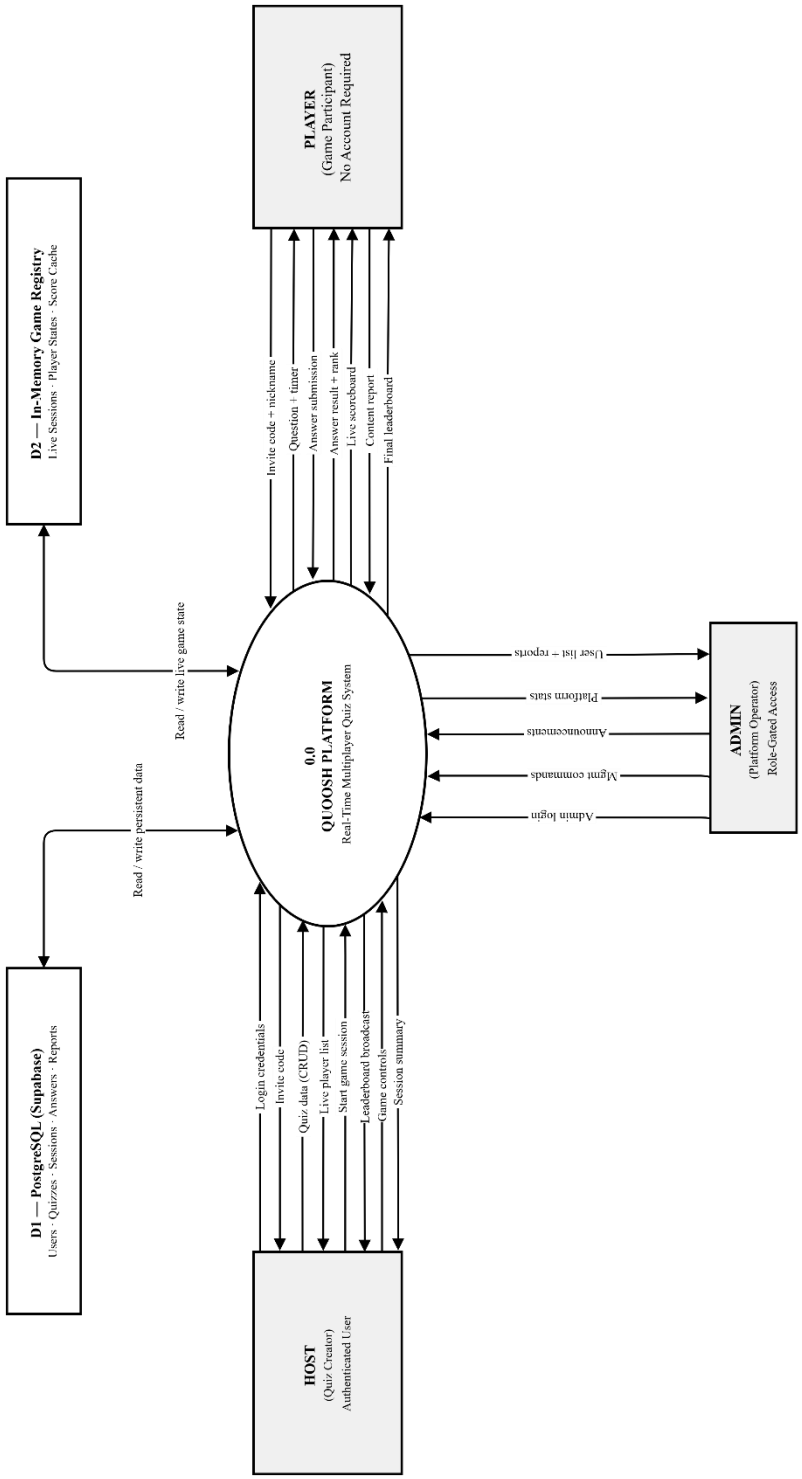

Fig. 3.4.1 — Context Diagram (Level 0 DFD) of the Quoosh Platform

3.4.2 Level 1 DFD

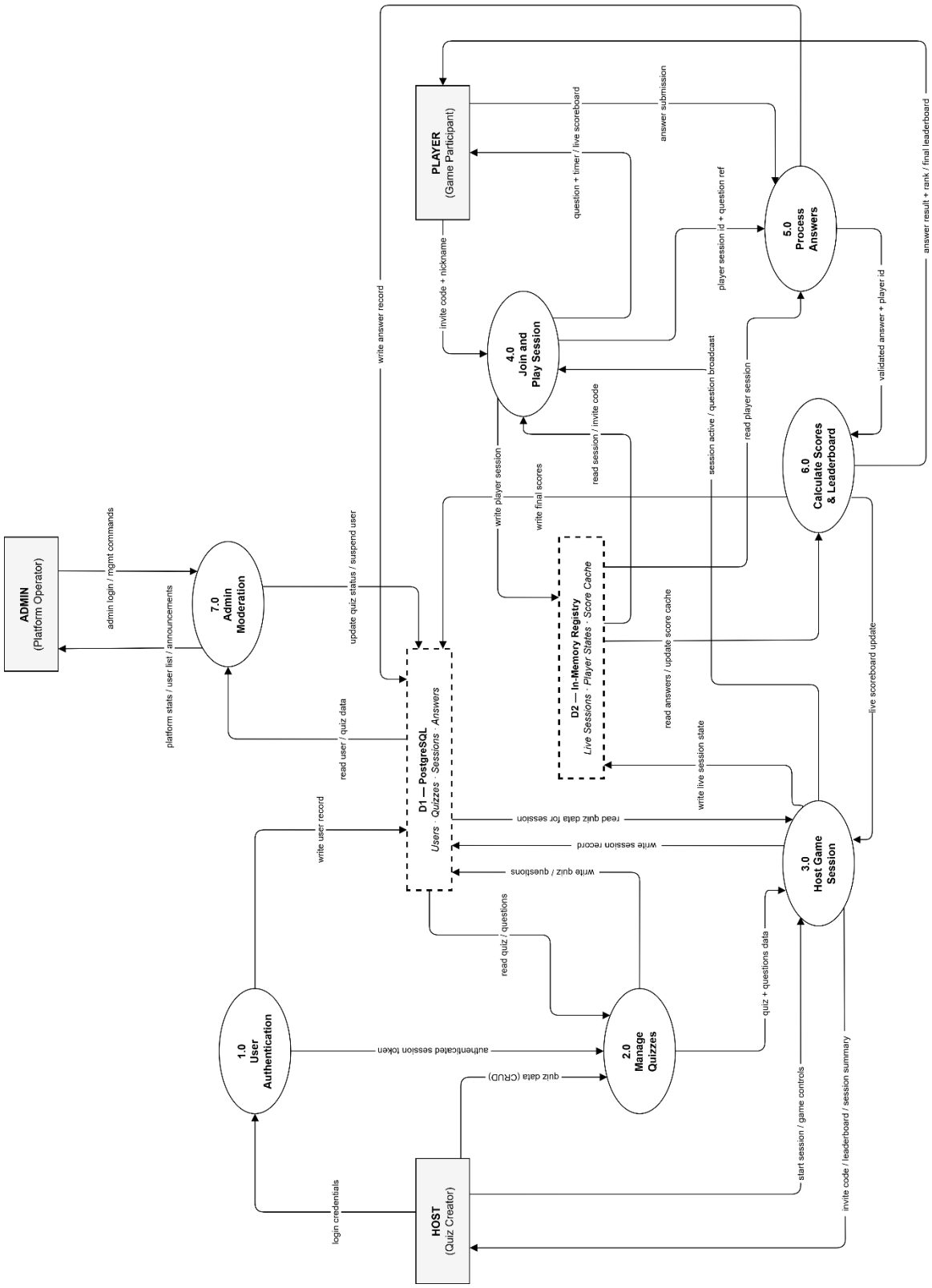

Fig. 3.3 – Level 1 DFD of Quoosh

3.5 UI Mock-ups / User Interface Design

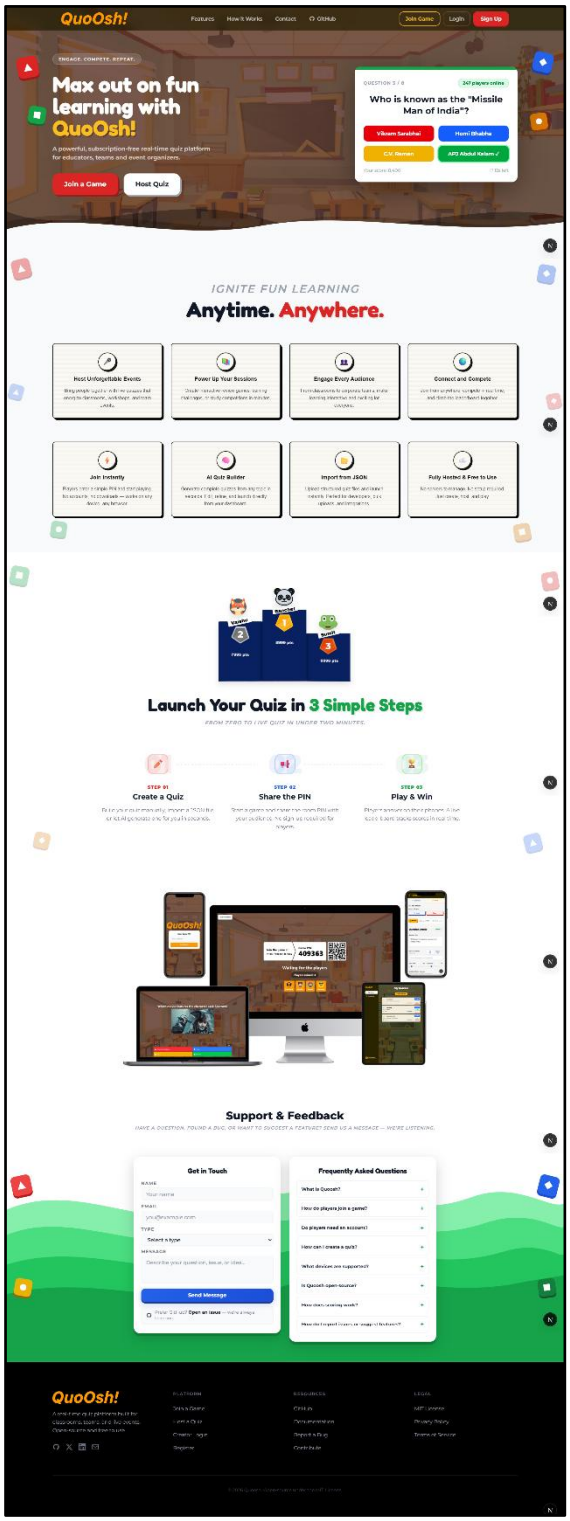

Fig. 3.5.1 – Landing Page of Quoosh

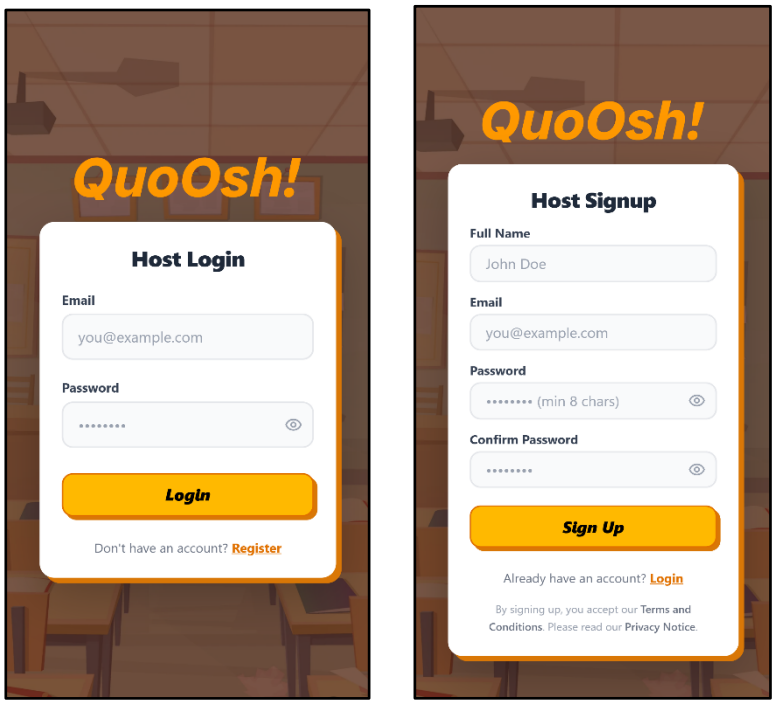

Fig. 3.5.2 – Host Login & Signup Screen

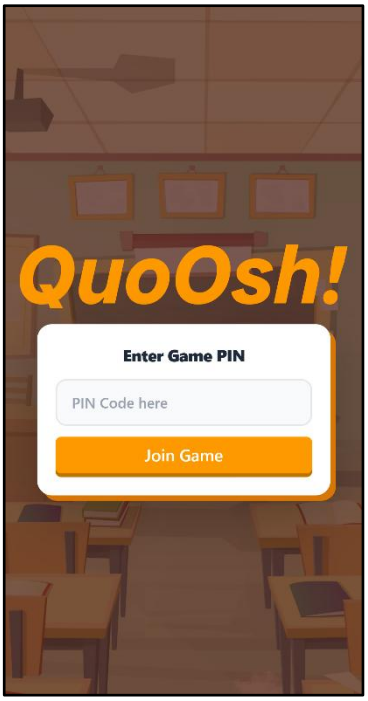

Fig. 3.5.3 – Player Join Screen

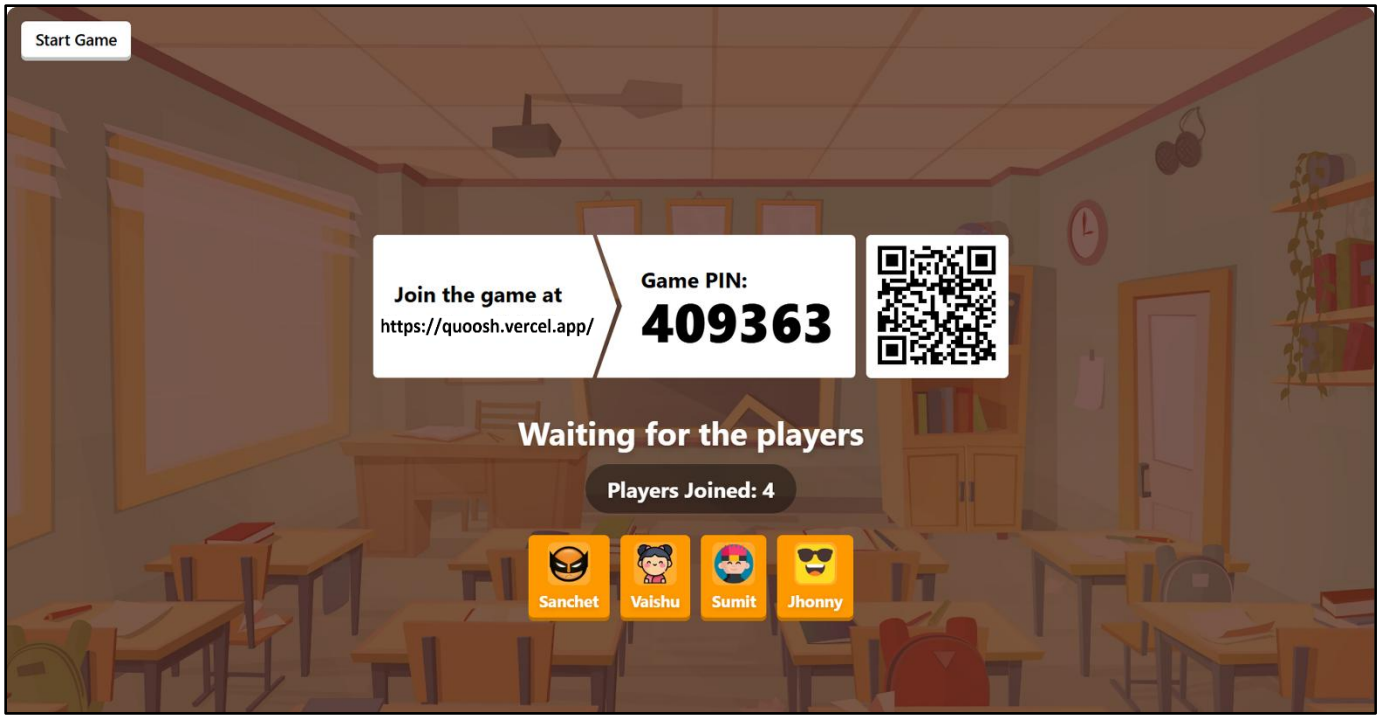

Fig. 3.5.4 – Game Lobby (Waiting Room)

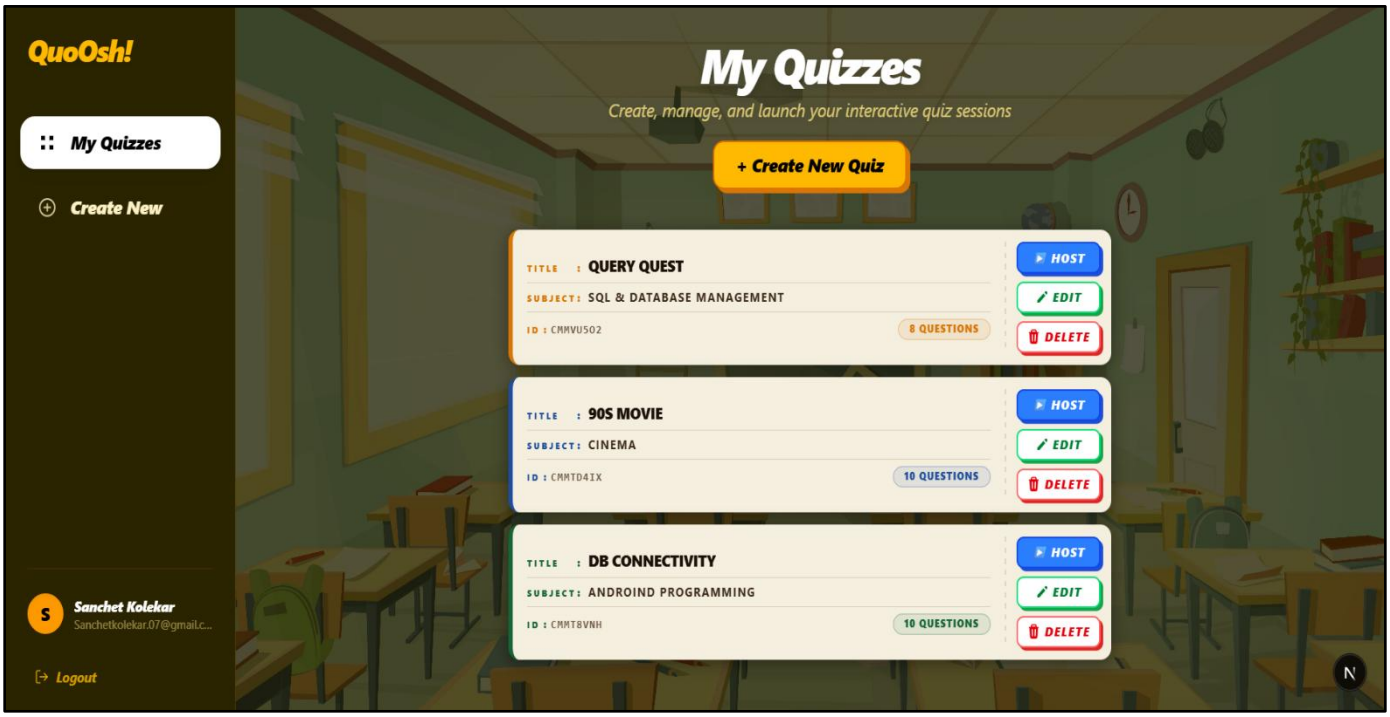

Fig. 3.5.5 – Host Dashboard

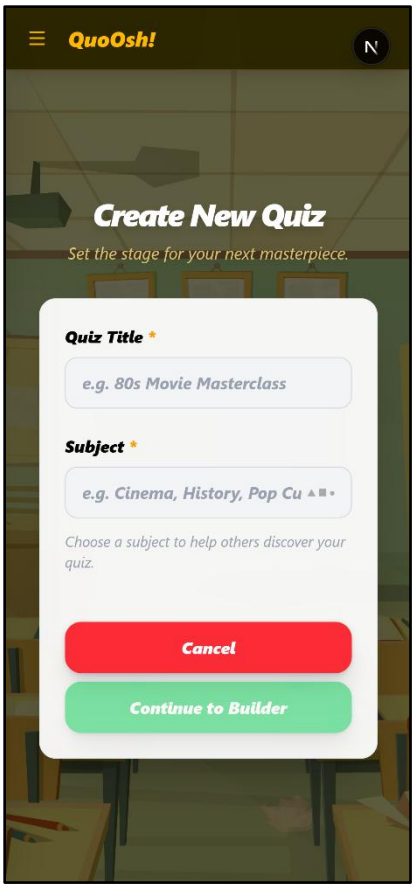

Fig. 3.5.6 – Create New Quiz Screen

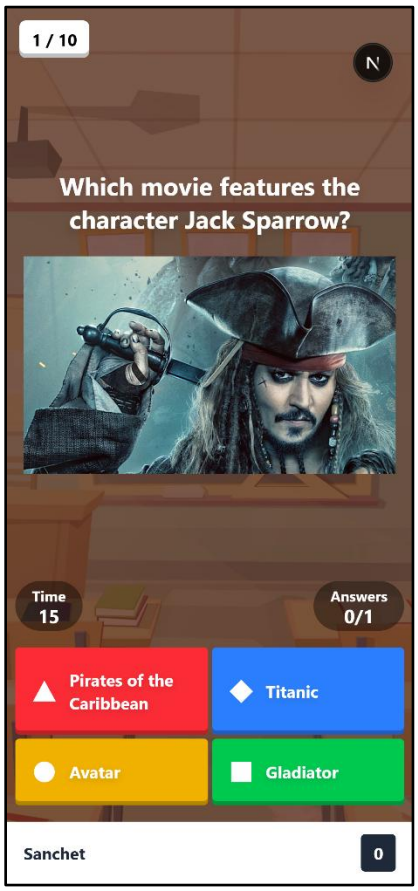

Fig. 3.5.7 – Live Quiz Interface

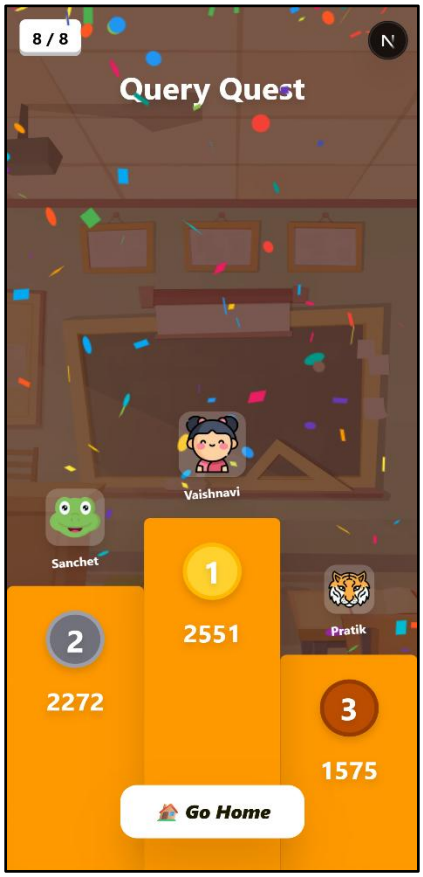

Fig. 3.5.8 – Leaderboard Screen

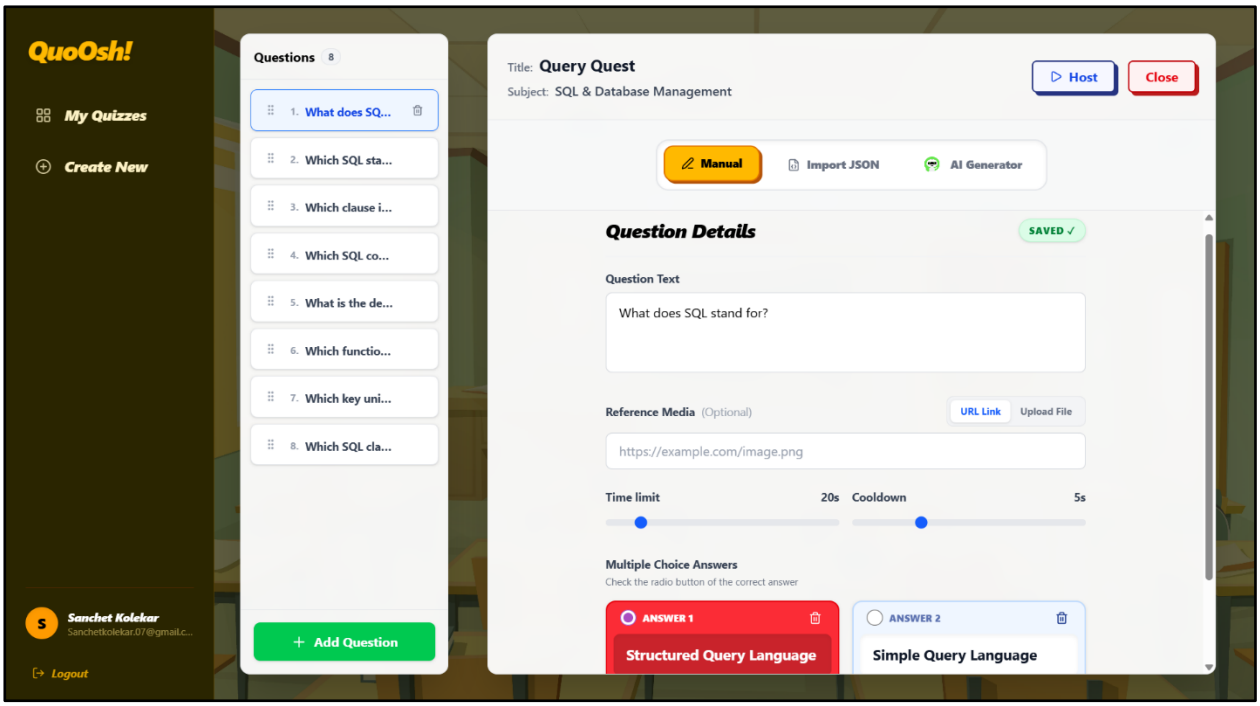

Fig. 3.5.9 – Quiz Creation Interface

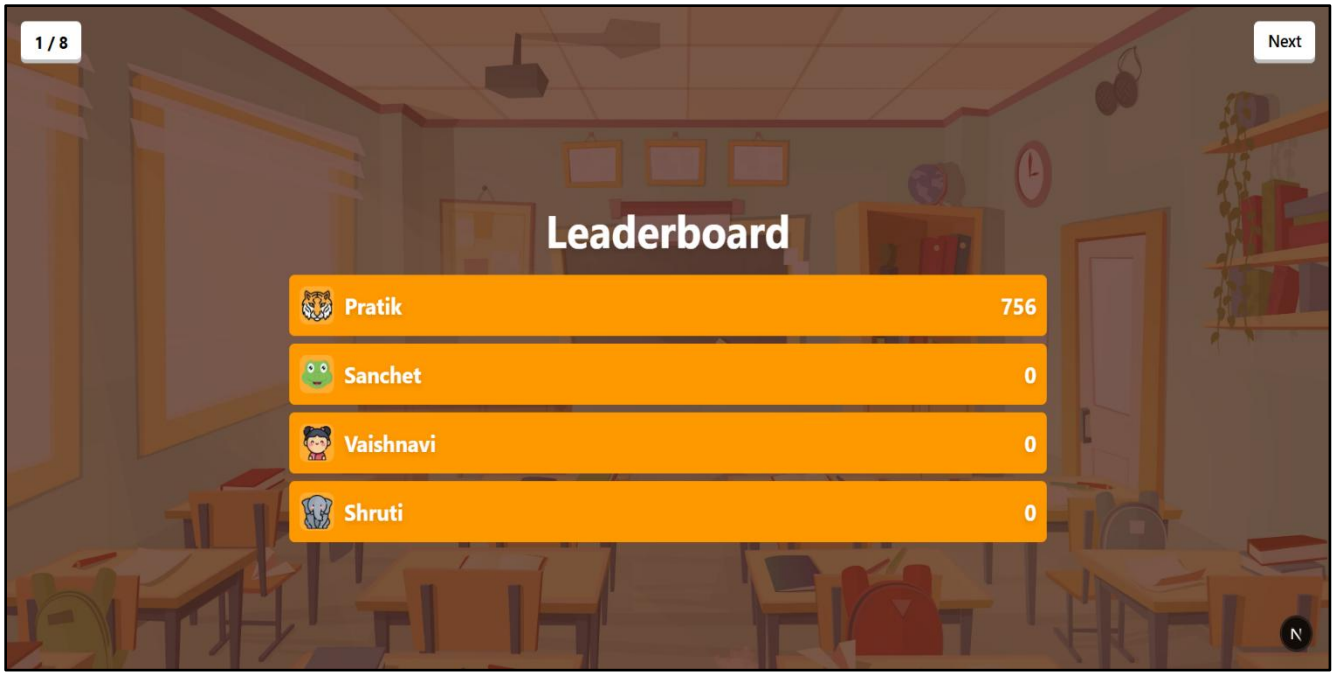

Fig. 3.5.10 – Quiz In Between Leaderboard

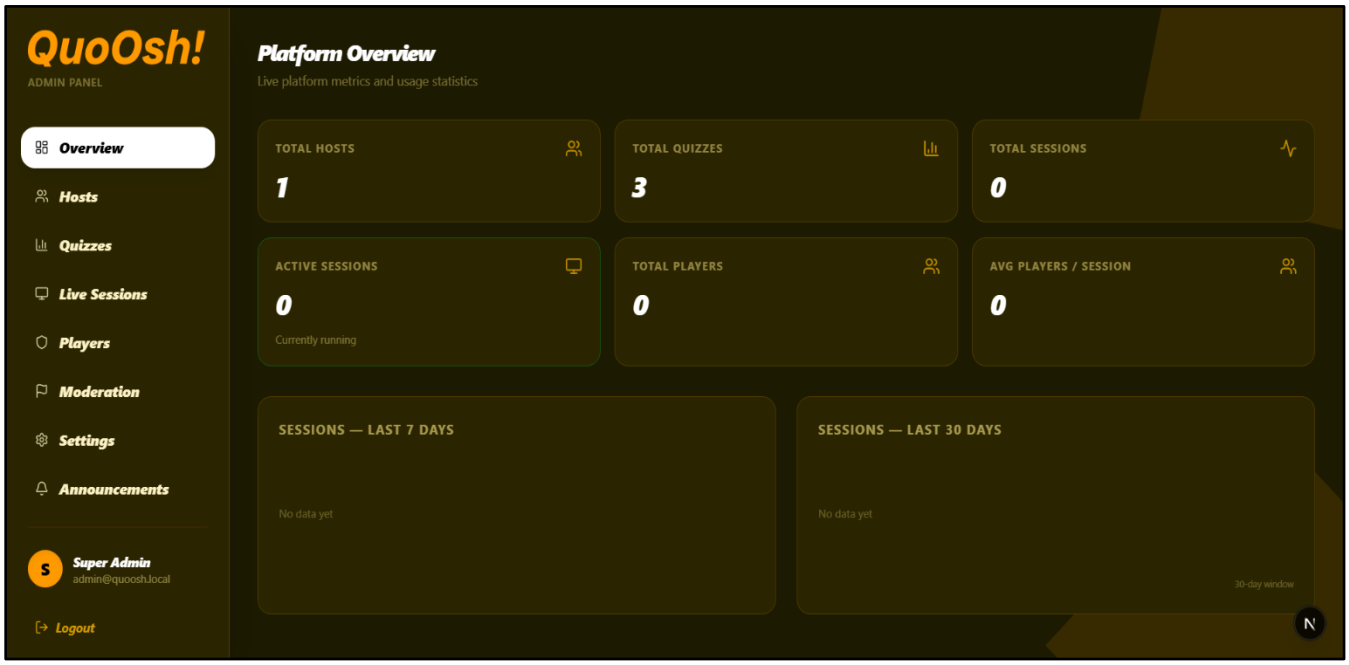

Fig. 3.5.11 – Admin Dashboard

4. Operating Environment – Hardware and Software

4.1 Hardware Environment

a) Client-Side (Web Platform – Phase 1)

- Any modern Smartphone, Tablet, Desktop, or Laptop
- Minimum 2 GB RAM recommended
- Modern web browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or Apple Safari
- Stable internet connectivity required for real-time WebSocket communication

b) Server-Side (Cloud Infrastructure)

i. Web and API Server:

Node.js 14+ compatible cloud compute environment. Vercel (recommended) provides automatic scaling and global CDN. Minimum 512 MB RAM for Next.js processes.

ii. Socket Server:

Separate Node.js 20 process deployed on Railway. Requires persistent WebSocket connection support serverless platforms are not compatible. Minimum 256 MB RAM.

iii. Database Server:

PostgreSQL 17 managed instance on Neon (Singapore region). Connection pooling enabled. Minimum 1 GB storage.

iv. Redis:

Upstash serverless Redis (Mumbai region) for sliding-window rate limiting.

4.2 Software Environment

Layer	Technology	Version	Purpose
Frontend	Next.js	16.1.5	App framework, SSR, App Router, API routes
Frontend	React	19.2.4	UI component library
Frontend	Tailwind CSS	4.2.2	Utility-first CSS framework
Frontend	Zustand	5.0.10	Client-side game state management
Frontend	Socket.IO Client	4.8.3	WebSocket client for real-time gameplay
Frontend	Motion	12.29.2	Animation library
Frontend	@dnd-kit	6.3.1	Drag-and-drop question reordering
Frontend	React Hook Form	7.71.2	Form state management
Backend API	Auth.js (next-auth)	5.0.0-beta.30	Authentication with JWT sessions
Backend API	bcryptjs	3.0.3	Password hashing (cost factor 12)
Backend API	Prisma ORM	5.22.0	Type-safe DB access and migrations
Backend API	@upstash/ratelimit	2.0.8	Sliding-window rate limiting
Backend API	@upstash/redis	1.37.0	Upstash Redis client
Backend API	Resend	6.9.4	Transactional email (contact form)
Backend API	Vercel AI SDK	6.0.116	AI SDK primitives
Backend API	@ai-sdk/google	3.0.43	Google Gemini AI quiz generation
Game Server	Socket.IO	4.8.3	WebSocket server for live game events
Game Server	Node.js	20+	Server-side JavaScript runtime
Database	PostgreSQL	17	Relational database (hosted on Neon)

Shared	TypeScript	5.9.3	Static typing across all packages
Shared	Zod	4.3.6	Runtime schema validation
DevTools	pnpm	9.x	Package manager with workspace support
DevTools	esbuild	0.27.2	Socket server production bundler

5. Deployment architecture

The Quoosh platform follows a distributed cloud architecture using free-tier services to ensure scalability, reliability, and real-time performance. It consists of three main components: the Web Application Layer, the Real-Time Game Server, and the Database Layer, augmented by a Redis layer for rate limiting.

5.1 Web Application – Vercel

The Next.js 16.1.5 web application is deployed on Vercel with Next.js standalone output mode, providing automatic scaling, global CDN distribution, and zero-configuration deployment.

Live URL: **https://quoosh.vercel.app/**

Responsibilities:

- User authentication and session management via Auth.js v5.
- Host dashboard for quiz creation, editing, and management.
- Admin panel for platform governance.
- All 22 REST API endpoints for data operations.
- Database interaction through Prisma ORM v5.22.0.
- Rate limiting via Upstash Redis (@upstash/ratelimit).

5.2 Real-Time Game Engine – Railway

The multiplayer quiz gameplay requires persistent WebSocket connections. Since serverless platforms do not support long-lived connections, the Socket.IO v4 server is deployed separately on Railway as a dedicated Node.js 20 process, bundled using esbuild.

Live URL: **https://quoosh-production.up.railway.app**

Responsibilities:

- Managing live game sessions in memory via the Registry singleton.

- Maintaining player and host WebSocket connections.
- Broadcasting real-time score updates and question timers.
- Managing leaderboard state and session lifecycle.
- Admin namespace (/admin) for live session monitoring.

5.3 Database – Supabase (PostgreSQL)

The system uses a managed PostgreSQL 17 database hosted on Neon, a serverless PostgreSQL platform. The Singapore (ap-southeast-1) region was selected to minimize latency for Indian users. Neon provides connection pooling, automated backups, and a web-based management console.

Data stored:

- Users, Quizzes, Questions.
- Quiz Sessions and Player Sessions.
- Player Answers and Scores.
- Reports, Announcements, and Platform Settings.
- 4 versioned Prisma migrations managing the complete schema lifecycle.

5.4 Rate Limiting — Upstash Redis

Upstash Redis (Mumbai, ap-south-1 region) provides serverless Redis for sliding-window rate limiting. Registration is limited to 5 requests per 10 minutes per IP address. Login attempts are limited to 10 attempts per 5 minutes per email address. Rate limiting is automatically disabled if the Upstash environment variables are not configured.

5.5 Deployment Architecture Diagram

```
┌──────────────────────────────────────────────────────────┐
│            Users (Hosts / Players / Admins)              │
│                          |                               │
│                          v                               │
│     +-----------------------------------------+          │
│     |            Vercel Cloud Platform        |          │
│     |  Next.js 16.1.5 Web Application + APIs  |          │
│     |        https://quoosh.vercel.app        |          │
│     +--------+--------------------------+-------+         │
│              |                          |                 │
│           Prisma/TLS                 WebSocket            │
│              |                          |                 │
│              v                          v                 │
│   +----------------------+   +--------------------------+ │
│   |  Neon PostgreSQL 17  |   | Railway Cloud Platform   | │
│   |  Singapore Region    |   | Socket.IO v4 Game Server | │
│   +----------------------+   +--------------------------+ │
│                    |                                      │
│                    v                                      │
│   +-------------------------------------------+           │
│   |  Upstash Redis (Mumbai) -- Rate Limiting  |           │
│   +-------------------------------------------+           │
└──────────────────────────────────────────────────────────┘
```

Fig. 5.1 – Distributed Cloud Deployment Architecture of Quoos

5.6 Advantages of this Deployment Architecture

- Entirely free-tier cloud infrastructure — no upfront costs.
- Automatic scaling and global CDN distribution through Vercel with Next.js standalone output.
- Dedicated real-time communication server with persistent WebSocket support on Railway.
- Serverless PostgreSQL 17 with connection pooling and automated backups via Neon.

- Serverless Redis with sliding-window rate limiting via Upstash (Mumbai, closest to Indian users).
- Architectural separation ensures high-frequency gameplay events on the socket server do not degrade REST API performance.
- Automatic HTTPS enforced on all four deployment platforms (Vercel, Railway, Neon, Upstash).

6. Drawbacks and Limitations

- **In-Memory Game State:** Active game sessions are stored only in server memory. If the server crashes or restarts, ongoing sessions are lost. Old sessions are automatically cleared after a short timeout.
- **Limited Scalability:** The system runs on a single server process, which restricts handling a large number of concurrent users. A distributed system (e.g., Redis) would be needed for scaling.
- **Tight System Coupling:** Quiz data is directly passed from the web API to the socket server, creating strong dependency between components.
- **Static CORS Configuration:** Allowed domains are hardcoded, requiring code changes for deployment on different domains.
- **Browser-Only Platform:** The application is accessible only via web browsers, with no native mobile app support.
- **AI Dependency:** Quiz generation relies on the Google Gemini API, which may have rate limits or availability issues.
- **Network Dependency:** Smooth real-time gameplay requires a stable, low-latency internet connection.
- **Admin Security Exposure:** The admin socket secret is exposed in the frontend to enable dashboard communication.

7. Future Scope

The Quoosh architecture provides a scalable foundation for future enhancements across accessibility, performance, and feature expansion.

- **Platform Expansion:** Development of native Android and iOS applications to improve accessibility.
- **Scalability Improvements:** Integration of distributed systems (e.g., Redis Pub/Sub) to support large-scale concurrent users.

- **Fault Tolerance:** Implementation of session persistence to enable recovery of game state after server failures.
- **Enhanced Content Support:** Addition of audio and video-based questions with cloud storage integration.
- **Analytics and Insights:** Advanced dashboards for tracking performance, engagement, and question analysis.
- **Collaborative Gameplay:** Introduction of team-based modes with shared scoring and participation.
- **Public Quiz Library:** Support for sharing and discovering reusable quiz templates.
- **Competitive Features:** Implementation of tournament modes with brackets and scheduling.
- **Database Persistence:** Storing session and player data in the database for audit and reporting.
- **Security Enhancements:** Implementation of a full Content Security Policy (CSP) for improved application security.

8. Conclusion

The Quoosh project successfully delivers a scalable, full-stack, real-time multiplayer quiz platform integrating quiz creation, AI-assisted content generation, live gameplay, and administrative control within a unified system.

Built using modern technologies such as Next.js, Socket.IO, Prisma, and PostgreSQL in a monorepo architecture, the platform ensures efficient real-time communication and structured data management. It is fully deployed and accessible online, demonstrating practical implementation.

The system meets its key objectives, including role-based access control, real-time game synchronization, secure authentication, rate limiting, and an admin dashboard for platform management.

Overall, Quoosh showcases the effective use of modern web technologies to create an interactive, secure, and extensible platform suitable for educational and collaborative environments.

9. Bibliography (Reference)

9.1 Research Papers & Academic Sources

1. **Chans, G. U. & Castro, M. M.**, "*Using Kahoot! as a Formative Assessment Tool in Pre- and Post-Reading Activities: A Study of EFL Students' Perceptions*", English Language Teaching, Vol. 14, No. 1, 2021.

2. **Wang, A. I. & Lieberoth**, A., "*The Effect of Points and Audio on Concentration, Engagement, Enjoyment, Learning, Motivation, and Classroom Dynamics Using Kahoot!*", European Conference on Games Based Learning, 2016.

3. **Subhash, S. & Cudney**, E. A., "*Gamified Learning in Higher Education: A Systematic Review of the Literature*", Computers in Human Behavior, Vol. 87, 2018.

4. **Deterding, S. et al.**, "*From Game Design Elements to Gamefulness: Defining Gamification*", Proceedings of MindTrek '11, ACM, 2011.

9.2 Internet Tools & Documentation

1. Next.js, *Next.js Documentation*, https://nextjs.org/docs
2. Socket.IO, *Socket.IO Documentation*, https://socket.io/docs/v4
3. Prisma, *Prisma ORM Documentation*, https://www.prisma.io/docs/orm
4. Auth.js, *Auth.js Documentation*, https://authjs.dev/getting-started
5. PostgreSQL Global Development Group, *PostgreSQL Documentation*, https://www.postgresql.org/docs/current
6. Neon, *Neon Documentation*, https://neon.tech/docs
7. Upstash, *Upstash Redis Documentation*, https://upstash.com/docs/redis
8. Google, *Google Generative AI SDK*, https://ai-sdk.dev/providers/ai-sdk-providers/google-generative-ai
9. Resend, *Resend Email API Documentation*, https://resend.com/docs
10. pmndrs, *Zustand GitHub Repository*, https://github.com/pmndrs/zustand
11. pnpm, *pnpm Workspaces Documentation*, https://pnpm.io/workspaces
12. *Quoosh: Real-time Quiz Platform*, Source Code https://github.com/Sanchet237/Quoosh